University of British Columbia, Vancouver
Department of Computer Science

CPSC 304 Project Cover Page

Milestone #: 2

Date: 10/22/2025

Group Number: 84

Name	Student Number	CS Alias (Userid)	Preferred E-mail Address
Albertus Ivan Wijaya	38138061	s8u4e	wialbertusivan@gmail.com
Jose Damara Hadriana	55546568	z7q2a	jhadrian@student.ubc.ca
Kenneth Yonathan Arianto	84225986	q9y2m	kennethyo2005@gmail.com

By typing our names and student numbers in the above table, we certify that the work in the attached assignment was performed solely by those whose names and student IDs are included above. (In the case of Project Milestone 0, the main purpose of this page is for you to let us know your e-mail address, and then let us assign you to a TA for your project supervisor.)

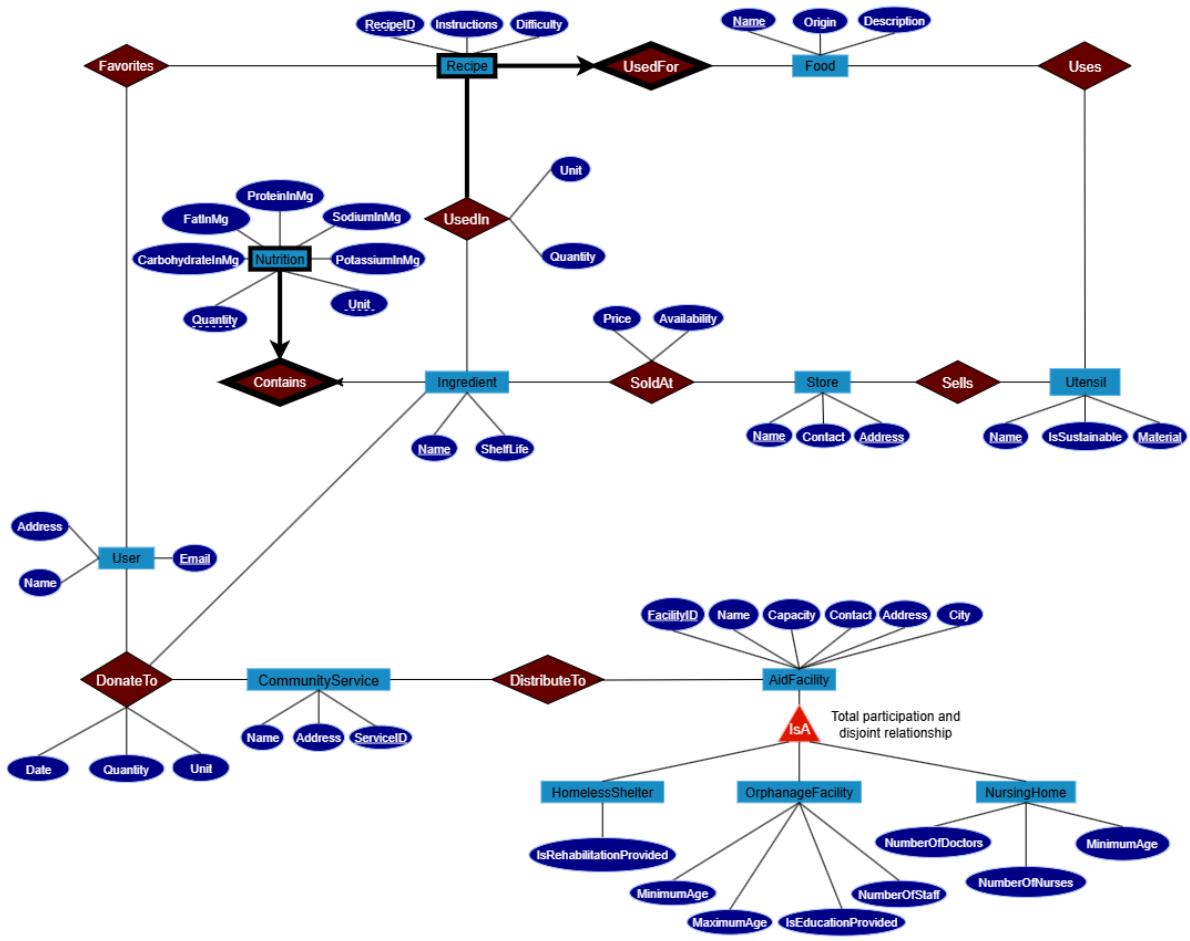
In addition, we indicate that we are fully aware of the rules and consequences of plagiarism, as set forth by the Department of Computer Science and the University of British Columbia

University of British Columbia, Vancouver
Department of Computer Science

Brief Summary of The Project

2. This project manages leftover food ingredients by helping users repurpose them into new dishes or donate them to community services. The system supports nutrient lookup per weight, recipe search, nearby store/utensil discovery, and recording donations (who donated what, when, and where).

3.



We made a few changes to the ER diagram based on the TA's feedback from Milestone 1 submission. Specifically, we modified the primary key of the Store entity to be a combination of two attributes consisting of (Name, Address). Additionally, we introduced a ternary relationship between Ingredient, User, and CommunityService to capture the donation process better. This relationship has many-to-many-to-many cardinality with partial participation for all three entities. Lastly, we moved the attribute of the Contains relationship to the Nutrition weak entity to represent the Nutrition better (need to present the quantity and unit attribute to correctly presenting other Nutrition attributes).

4. Here are the schema derived from the ER diagram above, the Primary Key provided will automatically be UNIQUE and NOT NULL:

- User(Name: VARCHAR(50), Address: VARCHAR(100), Email: VARCHAR(50))
  PK: (Email)
  CK: (Name, Address), (Email)
  Constraints: NOT NULL for all attributes

- Store(Name: VARCHAR(50), Address: VARCHAR(100), Contact: VARCHAR(50))
  PK: (Name, Address)
  Constraints: NOT NULL for all attributes

- Utensil(Name: VARCHAR(50), IsSustainable: BOOLEAN, Material: VARCHAR(30))
  PK: (Name, Material)

- Ingredient(Name: VARCHAR(50), ShelfLife: INT)
  PK: (Name)

- Nutrition(FatInMg: FLOAT, ProteinInMg: FLOAT, CarbohydrateInMg: FLOAT, SodiumInMg: FLOAT, PotassiumInMg: FLOAT, IngredientName: VARCHAR(50), Quantity: FLOAT, Unit: VARCHAR(10))
  PK: (IngredientName, Quantity, Unit)
  FK: (IngredientName) REFERENCES Ingredient(Name)

- Food(Name: VARCHAR(50), Origin: VARCHAR(30), Description: VARCHAR(200))
  PK: (Name)

- Recipe(RecipeID: INT, Instructions: VARCHAR(1000), Difficulty: VARCHAR(20), FoodName: VARCHAR(50))
  PK: (RecipeID, FoodName)
  FK: (FoodName) REFERENCES Food(Name)

- CommunityService(ServiceID: INT, Name: VARCHAR(50), Address: VARCHAR(100))
  PK: (ServiceID)
  CK: (Name, Address), (ServiceID)
  Constraints: NOT NULL for all attributes

- AidFacility(FacilityID: INT, Name: VARCHAR(50), Capacity: INT, Contact: VARCHAR(50), Address: VARCHAR(100), City: VARCHAR(50))
  PK: (FacilityID)
  CK: (Name, Address), (FacilityID)

Constraints: NOT NULL for all attributes

- HomelessShelter(FacilityID: INT, IsRehabilitationProvided: BOOLEAN)
  PK: (FacilityID)
  FK: (FacilityID) REFERENCES AidFacility(FacilityID)
  Constraints: Total and disjoint ISA participation

- OrphanageFacility(FacilityID: INT, MinimumAge: INT, MaximumAge: INT, IsEducationProvided: BOOLEAN, NumberOfStaff: INT)
  PK: (FacilityID)
  FK: (FacilityID) REFERENCES AidFacility(FacilityID)
  Constraints: Total and disjoint ISA participation

- NursingHome(FacilityID: INT, NumberOfDoctors: INT, NumberOfNurses: INT, MinimumAge: INT)
  PK: (FacilityID)
  FK: (FacilityID) REFERENCES AidFacility(FacilityID)
  Constraints: Total and disjoint ISA participation

- DonateTo(UserEmail: VARCHAR(50), IngredientName: VARCHAR(50), ServiceID: INT, Date: DATE, Quantity: FLOAT, Unit: VARCHAR(10))
  PK: (UserEmail, IngredientName, ServiceID)
  FK1: (UserEmail) REFERENCES User(Email)
  FK2: (IngredientName) REFERENCES Ingredient(Name)
  FK3: (ServiceID) REFERENCES CommunityService(ServiceID)
  Constraints: All partial participation; many-to-many-to-many relationship

- DistributeTo(ServiceID: INT, FacilityID: INT)
  PK: (ServiceID, FacilityID)
  FK1: (ServiceID) REFERENCES CommunityService(ServiceID)
  FK2: (FacilityID) REFERENCES AidFacility(FacilityID)
  Constraints: All partial participation; many-to-many relationship

- SoldAt(IngredientName: VARCHAR(50), StoreName: VARCHAR(50), StoreAddress: VARCHAR(100), Price: FLOAT, Availability: BOOLEAN)
  PK: (IngredientName, StoreName, StoreAddress)
  FK1: (IngredientName) REFERENCES Ingredient(Name)
  FK2: (StoreName, StoreAddress) REFERENCES Store(Name, Address)
  Constraints: All partial participation; many-to-many relationship

- Sells(StoreName: VARCHAR(50), StoreAddress: VARCHAR(100), UtensilName: VARCHAR(50), UtensilMaterial: VARCHAR(50))
  PK: (StoreName, StoreAddress, UtensilName, UtensilMaterial)

FK1: (StoreName, StoreAddress) REFERENCES Store(Name, Address)
FK2: (UtensilName, UtensilMaterial) REFERENCES Utensil(Name, Material)
Constraints: All partial participation; many-to-many relationship

- Uses(FoodName: VARCHAR(50), UtensilName: VARCHAR(50), UtensilMaterial: VARCHAR(50))
  PK: (FoodName, UtensilName, UtensilMaterial)
  FK1: (FoodName) REFERENCES Food(Name)
  FK2: (UtensilName, UtensilMaterial) REFERENCES Utensil(Name, Material)
  Constraints: All partial participation; many-to-many relationship

- UsedIn(IngredientName: VARCHAR(50), RecipeID: INT, FoodName: VARCHAR(50), Unit: VARCHAR(20), Quantity: FLOAT)
  PK: (IngredientName, RecipeID, FoodName)
  FK1: (IngredientName) REFERENCES Ingredient(Name)
  FK2: (RecipeID, FoodName) REFERENCES Recipe(RecipeID, FoodName)
  Constraints: Total participation for recipe, and Partial participation for Ingredient; many-to-many relationship

- Favorites(UserEmail: VARCHAR(50), RecipeID: INT, FoodName: VARCHAR(50))
  PK: (UserEmail, RecipeID, FoodName)
  FK1: (UserEmail) REFERENCES User(Email)
  FK2: (RecipeID, FoodName) REFERENCES Recipe(RecipeID, FoodName)
  Constraints: All partial participation; many-to-many relationship

5.) The Functional Dependencies (FDs) of each relation:

- User:
  - Email -> Name, Address (PK FD)
  - Name, Address -> Email (CK FD)

- Store:
  - Name, Address -> Contact

- Utensil:
  - Name, Material -> IsSustainable

- Ingredient:
  - Name -> ShelfLife

- Nutrition (weak entity):

- IngredientName, Quantity, Unit -> FatInMg, ProteinInMg, CarbohydrateInMg, SodiumInMg, PotassiumInMg.
- IngredientName -> FatInMg, ProteinInMg, CarbohydrateInMg, SodiumInMg, PotassiumInMg.

- Food:
  - Name -> Origin, Description

- Recipe (Weak Entity):
  - RecipeID, FoodName -> Instructions, Difficulty

- CommunityService:
  - ServiceID -> Name, Address
  - Name, Address -> ServiceID (CK FD)

- AidFacility:
  - FacilityID -> Name, Capacity, Contact, Address, City (PK FD)
  - Name, Address -> FacilityID, Contact, Address, City (CK FD)

- HomelessShelter:
  - FacilityID -> IsRehabilitationProvided

- OrphanageFacility:
  - FacilityID -> MinimumAge, MaximumAge, IsEducationProvided, NumberOfStaff

- NursingHome:
  - FacilityID -> NumberOfDoctors, NumberOfNurses, MinimumAge

- DonateTo (Ternary relationship):
  - UserEmail, IngredientName, ServiceID -> Date, Quantity, Unit

- DistributeTo:
  - No non-trivial FDs

- SoldAt:
  - IngredientName, StoreName, StoreAddress -> Price, Availability

- Sells:
  - No non-trivial FDs

- Uses:
  - No non-trivial FDs

- UsedIn:
    - IngredientName, RecipeID, FoodName -> Unit, Quantity

- Favorites:
    - No non-trivial FDs

6.) Here are the normalized tables:

- User(Name: VARCHAR(50), Address: VARCHAR(100), Email: VARCHAR(50))
  PK: Email
  CK: (Name, Address), Email
  Constraints: NOT NULL for all attributes
  FDs: Email -> Name, Address (PK FD)
  Name, Address -> Email (CK FD)

  Normalization: Table already in BCNF/3NF

- Store(Name: VARCHAR(50), Address: VARCHAR(100), Contact: VARCHAR(50))
  PK: (Name, Address)
  Constraints: NOT NULL for all attributes
  FDs: Name, Address -> Contact

  Normalization: Table already in BCNF/3NF

- Utensil(Name: VARCHAR(50), IsSustainable: BOOLEAN, Material: VARCHAR(30))
  PK: (Name, Material)
  FDs: Name, Material -> IsSustainable

  Normalization: Table already in BCNF/3NF

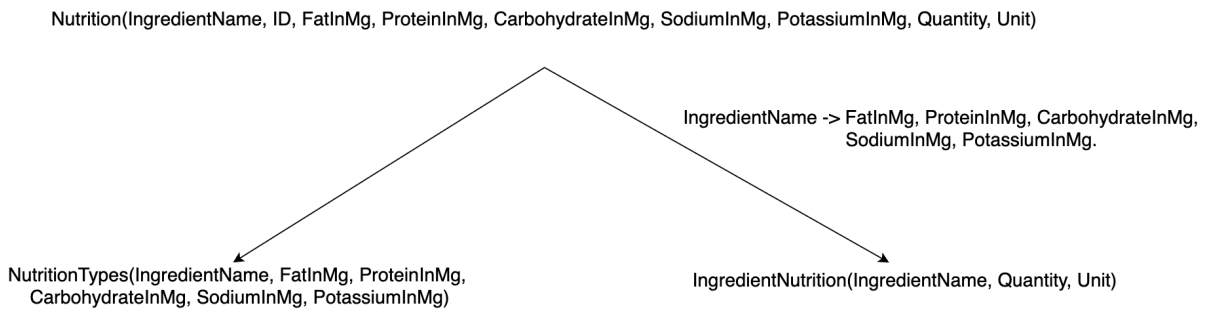
- Ingredient(Name: VARCHAR(50), ShelfLife: INT)
  PK: (Name)
  FDs: Name -> ShelfLife

  Normalization: Table already in BCNF/3NF

- Nutrition(FatInMg: FLOAT, ProteinInMg: FLOAT, CarbohydrateInMg: FLOAT, SodiumInMg: FLOAT, PotassiumInMg: FLOAT, IngredientName: VARCHAR(50), Quantity: FLOAT, Unit: VARCHAR(10))
  PK: (IngredientName, Quantity, Unit)
  FK: (IngredientName) REFERENCES Ingredient(Name)
  FDs: IngredientName, Quantity, Unit -> FatInMg, ProteinInMg, CarbohydrateInMg,

SodiumInMg, PotassiumInMg.
IngredientName -> FatInMg, ProteinInMg, CarbohydrateInMg, SodiumInMg, PotassiumInMg.

Normalization: Nutrition Table first FD violates BCNF/3NF, needs to do a BCNF decomposition



This results in two new, smaller tables called NutritionTypes and IngredientNutrition, which have relation schemas as below,

- ☐ NutritionTypes(IngredientName: VARCHAR(50), FatInMg: FLOAT, ProteinInMg: FLOAT, CarbohydrateInMg: FLOAT, SodiumInMg: FLOAT, PotassiumInMg: FLOAT)
  PK: (IngredientName)
  FK: (IngredientName) REFERENCES Ingredient(Name)
  FDs: IngredientName -> FatInMg, ProteinInMg, CarbohydrateInMg, SodiumInMg, PotassiumInMg.

  Where now, the NutritionTypes table satisfies BCNF/3NF with written FDs

- ☐ IngredientNutrition(IngredientName: VARCHAR(50), Quantity: FLOAT, Unit: VARCHAR(10))
  PK: (IngredientName, Quantity, Unit)
  FK: (IngredientName) REFERENCES Ingredient(Name)
  FDs: No non-trivial FDs

  Where now, the IngredientNutrition table automatically satisfies BCNF/3NF with written FDs (trivial FDs only)

- Food(Name: VARCHAR(50), Origin: VARCHAR(30), Description: VARCHAR(200))
  PK: (Name)

FDs: Name -> Origin, Description

Normalization: Table already in BCNF/3NF

- Recipe(RecipeID: INT, Instructions: VARCHAR(1000), Difficulty: VARCHAR(20), FoodName: VARCHAR(50))
  PK: (RecipeID, FoodName)
  FK: (FoodName) REFERENCES Food(Name)
  FDs: RecipeID, FoodName -> Instructions, Difficulty

  Normalization: Table already in BCNF/3NF

- CommunityService(ServiceID: INT, Name: VARCHAR(50), Address: VARCHAR(100))
  PK: (ServiceID)
  CK: (Name, Address), (ServiceID)
  Constraints: NOT NULL for all attributes
  FDs: ServiceID -> Name, Address
  Name, Address -> ServiceID (CK FD)

  Normalization: Table already in BCNF/3NF

- AidFacility(FacilityID: INT, Name: VARCHAR(50), Capacity: INT, Contact: VARCHAR(50), Address: VARCHAR(100), City: VARCHAR(50))
  PK: (FacilityID)
  CK: (Name, Address), (FacilityID)
  Constraints: NOT NULL for all attributes
  FDs: FacilityID -> Name, Capacity, Contact, Address, City (PK FD)
  Name, Address -> FacilityID, Contact, Address, City (CK FD)

  Normalization: Table already in BCNF/3NF

- HomelessShelter(FacilityID: INT, IsRehabilitationProvided: BOOLEAN)
  PK: (FacilityID)
  FK: (FacilityID) REFERENCES AidFacility(FacilityID)
  Constraints: Total and disjoint ISA participation
  FDs: FacilityID -> IsRehabilitationProvided

  Normalization: Table already in BCNF/3NF

- OrphanageFacility(FacilityID: INT, MinimumAge: INT, MaximumAge: INT, IsEducationProvided: BOOLEAN, NumberOfStaff: INT)
  PK: (FacilityID)
  FK: (FacilityID) REFERENCES AidFacility(FacilityID)

Constraints: Total and disjoint ISA participation
FDs: FacilityID -> MinimumAge, MaximumAge, IsEducationProvided, NumberOfStaff

Normalization: Table already in BCNF/3NF

- NursingHome(FacilityID: INT, NumberOfDoctors: INT, NumberOfNurses: INT, MinimumAge: INT)
  PK: (FacilityID)
  FK: (FacilityID) REFERENCES AidFacility(FacilityID)
  Constraints: Total and disjoint ISA participation
  FDs: FacilityID -> NumberOfDoctors, NumberOfNurses, MinimumAge

  Normalization: Table already in BCNF/3NF

- DonateTo(UserEmail: VARCHAR(50), IngredientName: VARCHAR(50), ServiceID: INT, Date: DATE, Quantity: FLOAT, Unit: VARCHAR(10))
  PK: (UserEmail, IngredientName, ServiceID)
  FK1: (UserEmail) REFERENCES User(Email)
  FK2: (IngredientName) REFERENCES Ingredient(Name)
  FK3: (ServiceID) REFERENCES CommunityService(ServiceID)
  Constraints: All partial participation; many-to-many-to-many relationship
  FDs: UserEmail, IngredientName, ServiceID -> Date, Quantity, Unit

  Normalization: Table already in BCNF/3NF

- DistributeTo(ServiceID: INT, FacilityID: INT)
  PK: (ServiceID, FacilityID)
  FK1: (ServiceID) REFERENCES CommunityService(ServiceID)
  FK2: (FacilityID) REFERENCES AidFacility(FacilityID)
  Constraints: All partial participation; many-to-many relationship
  FDs: No non-trivial FDs

  Normalization: Table already in BCNF/3NF

- SoldAt(IngredientName: VARCHAR(50), StoreName: VARCHAR(50), StoreAddress: VARCHAR(100), Price: FLOAT, Availability: BOOLEAN)
  PK: (IngredientName, StoreName, StoreAddress)
  FK1: (IngredientName) REFERENCES Ingredient(Name)
  FK2: (StoreName, StoreAddress) REFERENCES Store(Name, Address)
  Constraints: All partial participation; many-to-many relationship
  FDs: IngredientName, StoreName, StoreAddress -> Price, Availability

  Normalization: Table already in BCNF/3NF

- Sells(StoreName: VARCHAR(50), StoreAddress: VARCHAR(100), UtensilName: VARCHAR(50), UtensilMaterial: VARCHAR(50))
  PK: (StoreName, StoreAddress, UtensilName, UtensilMaterial)
  FK1: (StoreName, StoreAddress) REFERENCES Store(Name, Address)
  FK2: (UtensilName, UtensilMaterial) REFERENCES Utensil(Name, Material)
  Constraints: All partial participation; many-to-many relationship
  FDs: No non-trivial FDs

  Normalization: Table already in BCNF/3NF

- Uses(FoodName: VARCHAR(50), UtensilName: VARCHAR(50), UtensilMaterial: VARCHAR(50))
  PK: (FoodName, UtensilName, UtensilMaterial)
  FK1: (FoodName) REFERENCES Food(Name)
  FK2: (UtensilName, UtensilMaterial) REFERENCES Utensil(Name, Material)
  Constraints: All partial participation; many-to-many relationship
  FDs: No non-trivial FDs

  Normalization: Table already in BCNF/3NF

- UsedIn(IngredientName: VARCHAR(50), RecipeID: INT, FoodName: VARCHAR(50), Unit: VARCHAR(20), Quantity: FLOAT)
  PK: (IngredientName, RecipeID, FoodName)
  FK1: (IngredientName) REFERENCES Ingredient(Name)
  FK2: (RecipeID, FoodName) REFERENCES Recipe(RecipeID, FoodName)
  Constraints: Total participation for recipe, and Partial participation for Ingredient; many-to-many relationship
  FDs: IngredientName, RecipeID, FoodName -> Unit, Quantity

  Normalization: Table already in BCNF/3NF

- Favorites(UserEmail: VARCHAR(50), RecipeID: INT, FoodName: VARCHAR(50))
  PK: (UserEmail, RecipeID, FoodName)
  FK1: (UserEmail) REFERENCES User(Email)
  FK2: (RecipeID, FoodName) REFERENCES Recipe(RecipeID, FoodName)
  Constraints: All partial participation; many-to-many relationship
  FDs: No non-trivial FDs

  Normalization: Table already in BCNF/3NF

7.) Here are the SQL DDL statements required to create all the tables above:

Notes:
Oracle does NOT support "ON UPDATE CASCADE"
All tables normalized to BCNF/3NF.

```
-- 1) User
CREATE TABLE User (
   Name    VARCHAR(50)  NOT NULL,
   Address VARCHAR(100) NOT NULL,
   Email   VARCHAR(50),
   PRIMARY KEY (Email)
);

-- 2) Store
CREATE TABLE Store (
   Name    VARCHAR(50),
   Address VARCHAR(100),
   Contact VARCHAR(50)  NOT NULL,
   PRIMARY KEY (Name, Address)
);

-- 3) Utensil
CREATE TABLE Utensil (
   Name         VARCHAR(50),
   Material     VARCHAR(30),
   IsSustainable BOOLEAN,
   PRIMARY KEY (Name, Material)
);

-- 4) Ingredient
CREATE TABLE Ingredient (
   Name      VARCHAR(50),
   ShelfLife INT,
   PRIMARY KEY (Name)
);

-- 5) Food
CREATE TABLE Food (
   Name       VARCHAR(50),
   Origin     VARCHAR(30),
   Description VARCHAR(200),
   PRIMARY KEY (Name)
);
```

```
-- 6) CommunityService
CREATE TABLE CommunityService (
   ServiceID INT,
   Name      VARCHAR(50) NOT NULL,
   Address   VARCHAR(100) NOT NULL,
   PRIMARY KEY (ServiceID),
   UNIQUE (Name, Address)
);

-- 7) AidFacility (supertype)
CREATE TABLE AidFacility (
   FacilityID INT,
   Name       VARCHAR(50) NOT NULL,
   Capacity   INT,
   Contact    VARCHAR(50) NOT NULL,
   Address    VARCHAR(100) NOT NULL,
   City       VARCHAR(50),
   PRIMARY KEY (FacilityID),
   UNIQUE (Name, Address)
);

-- 8) HomelessShelter (subtype) -- cascade delete on supertype removal
CREATE TABLE HomelessShelter (
   FacilityID INT,
   IsRehabilitationProvided BOOLEAN,
   PRIMARY KEY (FacilityID),
   FOREIGN KEY (FacilityID) REFERENCES AidFacility(FacilityID)
     ON DELETE CASCADE
);

-- 9) OrphanageFacility (subtype) -- cascade delete on supertype removal
CREATE TABLE OrphanageFacility (
   FacilityID INT,
   MinimumAge INT,
   MaximumAge INT,
   IsEducationProvided BOOLEAN,
   NumberOfStaff INT,
   PRIMARY KEY (FacilityID),
   FOREIGN KEY (FacilityID) REFERENCES AidFacility(FacilityID)
     ON DELETE CASCADE
);

-- 10) NursingHome (subtype) -- cascade delete on supertype removal
```

```
CREATE TABLE NursingHome (
   FacilityID INT,
   NumberOfDoctors INT,
   NumberOfNurses  INT,
   MinimumAge      INT,
   PRIMARY KEY (FacilityID),
   FOREIGN KEY (FacilityID) REFERENCES AidFacility(FacilityID)
     ON DELETE CASCADE
);

-- Normalized tables for Nutrition Data

-- 11) NutritionTypes
-- Does not allow ingredient to be deleted if NutritionTypes still exist
CREATE TABLE NutritionTypes (
   IngredientName VARCHAR(50),
   FatInMg        FLOAT,
   ProteinInMg    FLOAT,
   CarbohydrateInMg FLOAT,
   SodiumInMg     FLOAT,
   PotassiumInMg  FLOAT,
   PRIMARY KEY (IngredientName),
   FOREIGN KEY (IngredientName) REFERENCES Ingredient(Name)
     ON DELETE NO ACTION
);

-- 12) IngredientNutrition
-- Does not allow ingredient to be deleted if IngredientNutrition still exist
CREATE TABLE IngredientNutrition (
   IngredientName VARCHAR(50),
   Quantity       FLOAT,
   Unit           VARCHAR(10),
   PRIMARY KEY (IngredientName, Quantity, Unit),
   FOREIGN KEY (IngredientName) REFERENCES Ingredient(Name)
     ON DELETE NO ACTION
);

-- Recipe-related entities

-- 13) Recipe (weak entity of Food)
– Deleting Food is restricted (NO ACTION) to protect weak entity relationship with recipes
CREATE TABLE Recipe (
```

```
    RecipeID    INT,
    FoodName    VARCHAR(50),
    Instructions VARCHAR(1000),
    Difficulty  VARCHAR(20),
    PRIMARY KEY (RecipeID, FoodName),
    FOREIGN KEY (FoodName) REFERENCES Food(Name)
      ON DELETE NO ACTION
);

-- Relationship tables

-- 14) DonateTo (ternary: User × Ingredient × CommunityService)
-- Preserve donation history: ON DELETE NO ACTION (NOT Cascade)
CREATE TABLE DonateTo (
    UserEmail      VARCHAR(50),
    IngredientName VARCHAR(50),
    ServiceID      INT,
    Date           DATE,
    Quantity       FLOAT,
    Unit           VARCHAR(10),
    PRIMARY KEY (UserEmail, IngredientName, ServiceID),
    FOREIGN KEY (UserEmail) REFERENCES User (Email)
      ON DELETE NO ACTION
     -- ON UPDATE CASCADE, -- (Oracle does not support it).
    FOREIGN KEY (IngredientName) REFERENCES Ingredient(Name)
      ON DELETE NO ACTION,
    FOREIGN KEY (ServiceID) REFERENCES CommunityService(ServiceID)
      ON DELETE NO ACTION
);

-- 15) DistributeTo (many-to-many: Service × Facility)
-- Keep records; no DELETE cascade
CREATE TABLE DistributeTo (
    ServiceID  INT,
    FacilityID INT,
    PRIMARY KEY (ServiceID, FacilityID),
    FOREIGN KEY (ServiceID) REFERENCES CommunityService(ServiceID)
      ON DELETE NO ACTION,
    FOREIGN KEY (FacilityID) REFERENCES AidFacility(FacilityID)
      ON DELETE NO ACTION
);

-- 16) SoldAt (Ingredient sold at a Store)
```

```
CREATE TABLE SoldAt (
   IngredientName VARCHAR(50),
   StoreName      VARCHAR(50),
   StoreAddress   VARCHAR(100),
   Price          FLOAT,
   Availability   BOOLEAN,
   PRIMARY KEY (IngredientName, StoreName, StoreAddress),
   FOREIGN KEY (IngredientName) REFERENCES Ingredient(Name)
     ON DELETE NO ACTION,
   FOREIGN KEY (StoreName, StoreAddress) REFERENCES Store(Name, Address)
     ON DELETE CASCADE
   --  ON UPDATE CASCADE -- (Oracle does not support it)
);

-- 17) Sells (Store sells Utensil)
-- If a Store is deleted, remove its Sells rows using DELETE CASCADE
-- If a Utensil is deleted, prevent to remove Sells rows using DELETE NO ACTION
CREATE TABLE Sells (
   StoreName       VARCHAR(50),
   StoreAddress    VARCHAR(100),
   UtensilName     VARCHAR(50),
   UtensilMaterial VARCHAR(50),
   PRIMARY KEY (StoreName, StoreAddress, UtensilName, UtensilMaterial),
   FOREIGN KEY (StoreName, StoreAddress) REFERENCES Store(Name, Address)
     ON DELETE CASCADE
   --  ON UPDATE CASCADE, -- (Oracle does not support it)
   FOREIGN KEY (UtensilName, UtensilMaterial) REFERENCES Utensil(Name, Material)
     ON DELETE NO ACTION
);



-- 18) Uses (Food needs Utensil)
CREATE TABLE Uses (
   FoodName       VARCHAR(50),
   UtensilName    VARCHAR(50),
   UtensilMaterial VARCHAR(50),
   PRIMARY KEY (FoodName, UtensilName, UtensilMaterial),
   FOREIGN KEY (FoodName) REFERENCES Food(Name)
     ON DELETE CASCADE,
   FOREIGN KEY (UtensilName, UtensilMaterial) REFERENCES Utensil(Name, Material)
     ON DELETE NO ACTION
);
```

```
-- 19) UsedIn (Ingredient used in Recipe)
-- If Recipe deleted, remove UsedIn rows (dependent on recipe)
-- If Ingredient deleted, keep UsedIn (NO ACTION) to preserve references
CREATE TABLE UsedIn (
   IngredientName VARCHAR(50),
   RecipeID      INT,
   FoodName      VARCHAR(50),
   Unit          VARCHAR(20),
   Quantity      FLOAT,
   PRIMARY KEY (IngredientName, RecipeID, FoodName),
   FOREIGN KEY (IngredientName) REFERENCES Ingredient(Name)
     ON DELETE NO ACTION,
   FOREIGN KEY (RecipeID, FoodName) REFERENCES Recipe(RecipeID, FoodName)
     ON DELETE CASCADE
);

-- 20) Favorites (User favorites a Recipe)
-- Remove favorites when user or recipe is removed
CREATE TABLE Favorites (
   UserEmail VARCHAR(50),
   RecipeID  INT,
   FoodName  VARCHAR(50),
   PRIMARY KEY (UserEmail, RecipeID, FoodName),
   FOREIGN KEY (UserEmail) REFERENCES User(Email)
     ON DELETE CASCADE,
   --  ON UPDATE CASCADE, -- (Oracle does not support it)
   FOREIGN KEY (RecipeID, FoodName) REFERENCES Recipe(RecipeID, FoodName)
     ON DELETE CASCADE
);
```

8.)

```
-- Users
INSERT INTO User(Name, Address, Email) VALUES ('Alice Johnson', '1234 Maple Street, Vancouver', 'alice.johnson@gmail.com');
INSERT INTO User(Name, Address, Email) VALUES ('Brian Lee', '22 King Edward Ave, Toronto', 'brian.lee@hotmail.com');
INSERT INTO User(Name, Address, Email) VALUES ('Catherine Smith', '89 Broadway Road, New York', 'catherine.smith@yahoo.com');
INSERT INTO User(Name, Address, Email) VALUES ('David Kim', '45 Granville St, Vancouver', 'david.kim@gmail.com');
INSERT INTO User(Name, Address, Email) VALUES ('Emma Brown', '300 Sunset Blvd, Los Angeles', 'emma.brown@outlook.com');
```

```
-- Store
INSERT INTO Store(Name, Address, Contact) VALUES ('GreenGrocer', '101 Market Street, Seattle', '+12065551234');
INSERT INTO Store(Name, Address, Contact) VALUES ('FreshMart', '505 Granville St, Vancouver', '+16045552345');
INSERT INTO Store(Name, Address, Contact) VALUES ('UrbanEats', '88 Robson St, Vancouver', '+16045559876');
INSERT INTO Store(Name, Address, Contact) VALUES ('DailyHarvest', '77 Queen St, Toronto', '+14165553456');
INSERT INTO Store(Name, Address, Contact) VALUES ('Nature's Basket', '42 Orchard Rd, San Francisco', '+14155556789');


-- Utensil
INSERT INTO Utensil(Name, Material, IsSustainable) VALUES ('Spoon', 'Stainless Steel', 1);
INSERT INTO Utensil(Name, Material, IsSustainable) VALUES ('Fork', 'Plastic', 0);
INSERT INTO Utensil(Name, Material, IsSustainable) VALUES ('Knife', 'Bamboo', 1);
INSERT INTO Utensil(Name, Material, IsSustainable) VALUES ('Chopsticks', 'Wood', 1);
INSERT INTO Utensil(Name, Material, IsSustainable) VALUES ('Plate', 'Ceramic', 1);


-- Ingredient
INSERT INTO Ingredient(Name, ShelfLife) VALUES ('Spinach', 5);
INSERT INTO Ingredient(Name, ShelfLife) VALUES ('Chicken Breast', 7);
INSERT INTO Ingredient(Name, ShelfLife) VALUES ('Beef', 10);
INSERT INTO Ingredient(Name, ShelfLife) VALUES ('Tomato', 6);
INSERT INTO Ingredient(Name, ShelfLife) VALUES ('Carrot', 12);


-- Food
INSERT INTO Food(Name, Origin, Description) VALUES ('Spaghetti Bolognese', 'Italy', 'A classic pasta dish with minced beef and tomato sauce.');
INSERT INTO Food(Name, Origin, Description) VALUES ('Chicken Stir Fry', 'China', 'Chicken and vegetables quickly fried with soy sauce.');
INSERT INTO Food(Name, Origin, Description) VALUES ('Beef Stew', 'England', 'Slow-cooked beef with carrots and potatoes.');
INSERT INTO Food(Name, Origin, Description) VALUES ('Caesar Salad', 'USA', 'Salad with romaine lettuce, croutons, and Caesar dressing.');
INSERT INTO Food(Name, Origin, Description) VALUES ('Vegetable Soup', 'France', 'A light broth with seasonal vegetables.');
```

```
-- CommunityService
INSERT INTO CommunityService(ServiceID, Name, Address) VALUES (1, 'Food for All', '200 Main St, Vancouver');
INSERT INTO CommunityService(ServiceID, Name, Address) VALUES (2, 'Helping Hands', '45 Oak St, Toronto');
INSERT INTO CommunityService(ServiceID, Name, Address) VALUES (3, 'Kind Kitchen', '99 Elm St, Seattle');
INSERT INTO CommunityService(ServiceID, Name, Address) VALUES (4, 'Harvest Aid', '10 Park Ave, New York');
INSERT INTO CommunityService(ServiceID, Name, Address) VALUES (5, 'Meals of Hope', '33 Sunset Rd, Los Angeles');

-- AidFacility
INSERT INTO AidFacility(FacilityID, Name, Capacity, Contact, Address, City) VALUES (1, 'Hope Shelter', 100, '+16045554321', '400 East 5th Ave', 'Vancouver');
INSERT INTO AidFacility(FacilityID, Name, Capacity, Contact, Address, City) VALUES (2, 'Safe Haven', 80, '+14165557890', '55 King St', 'Toronto');
INSERT INTO AidFacility(FacilityID, Name, Capacity, Contact, Address, City) VALUES (3, 'Comfort Home', 60, '+12065556789', '300 Pine St', 'Seattle');
INSERT INTO AidFacility(FacilityID, Name, Capacity, Contact, Address, City) VALUES (4, 'Starlight Orphanage', 120, '+12125559876', '200 Broadway', 'New York');
INSERT INTO AidFacility(FacilityID, Name, Capacity, Contact, Address, City) VALUES (5, 'Sunrise Nursing Home', 90, '+13105556789', '50 Sunset Blvd', 'Los Angeles');

-- HomelessShelter
INSERT INTO HomelessShelter(FacilityID, IsRehabilitationProvided) VALUES (1, 1);
INSERT INTO HomelessShelter(FacilityID, IsRehabilitationProvided) VALUES (2, 0);
INSERT INTO HomelessShelter(FacilityID, IsRehabilitationProvided) VALUES (3, 1);
INSERT INTO HomelessShelter(FacilityID, IsRehabilitationProvided) VALUES (4, 0);
INSERT INTO HomelessShelter(FacilityID, IsRehabilitationProvided) VALUES (5, 1);

-- OrphanageFacility
INSERT INTO OrphanageFacility(FacilityID, MinimumAge, MaximumAge, IsEducationProvided, NumberOfStaff) VALUES (4, 3, 18, 1, 25);
INSERT INTO OrphanageFacility(FacilityID, MinimumAge, MaximumAge, IsEducationProvided, NumberOfStaff) VALUES (5, 2, 17, 1, 30);
INSERT INTO OrphanageFacility(FacilityID, MinimumAge, MaximumAge, IsEducationProvided, NumberOfStaff) VALUES (3, 1, 15, 0, 20);
INSERT INTO OrphanageFacility(FacilityID, MinimumAge, MaximumAge, IsEducationProvided, NumberOfStaff) VALUES (2, 4, 16, 1, 22);
INSERT INTO OrphanageFacility(FacilityID, MinimumAge, MaximumAge, IsEducationProvided, NumberOfStaff) VALUES (1, 5, 18, 1, 18);
```

```
-- NursingHome
INSERT INTO NursingHome(FacilityID, NumberOfDoctors, NumberOfNurses, MinimumAge)
VALUES (5, 4, 10, 60);
INSERT INTO NursingHome(FacilityID, NumberOfDoctors, NumberOfNurses, MinimumAge)
VALUES (4, 3, 8, 65);
INSERT INTO NursingHome(FacilityID, NumberOfDoctors, NumberOfNurses, MinimumAge)
VALUES (3, 2, 6, 70);
INSERT INTO NursingHome(FacilityID, NumberOfDoctors, NumberOfNurses, MinimumAge)
VALUES (2, 5, 12, 55);
INSERT INTO NursingHome(FacilityID, NumberOfDoctors, NumberOfNurses, MinimumAge)
VALUES (1, 4, 9, 60);

-- NutritionTypes
INSERT INTO NutritionTypes(IngredientName, FatInMg, ProteinInMg, CarbohydrateInMg,
SodiumInMg, PotassiumInMg) VALUES ('Spinach', 400, 2900, 3600, 80, 560);
INSERT INTO NutritionTypes(IngredientName, FatInMg, ProteinInMg, CarbohydrateInMg,
SodiumInMg, PotassiumInMg) VALUES ('Chicken Breast', 3600, 31000, 0, 70, 256);
INSERT INTO NutritionTypes(IngredientName, FatInMg, ProteinInMg, CarbohydrateInMg,
SodiumInMg, PotassiumInMg) VALUES ('Beef', 15000, 26000, 0, 75, 318);
INSERT INTO NutritionTypes(IngredientName, FatInMg, ProteinInMg, CarbohydrateInMg,
SodiumInMg, PotassiumInMg) VALUES ('Tomato', 200, 900, 3900, 5, 290);
INSERT INTO NutritionTypes(IngredientName, FatInMg, ProteinInMg, CarbohydrateInMg,
SodiumInMg, PotassiumInMg) VALUES ('Carrot', 120, 900, 10000, 69, 320);

-- IngredientNutrition
INSERT INTO IngredientNutrition(IngredientName, Quantity, Unit) VALUES ('Spinach', 100, 'g');
INSERT INTO IngredientNutrition(IngredientName, Quantity, Unit) VALUES ('Chicken Breast',
150, 'g');
INSERT INTO IngredientNutrition(IngredientName, Quantity, Unit) VALUES ('Beef', 200, 'g');
INSERT INTO IngredientNutrition(IngredientName, Quantity, Unit) VALUES ('Tomato', 100, 'g');
INSERT INTO IngredientNutrition(IngredientName, Quantity, Unit) VALUES ('Carrot', 80, 'g');

-- Recipe
INSERT INTO Recipe(RecipeID, FoodName, Instructions, Difficulty) VALUES (1, 'Spaghetti
Bolognese', 'Boil pasta, cook minced beef with tomato sauce, and mix together.', 'Medium');
INSERT INTO Recipe(RecipeID, FoodName, Instructions, Difficulty) VALUES (2, 'Chicken Stir Fry',
'Fry chicken, add vegetables, and toss with soy sauce.', 'Easy');
INSERT INTO Recipe(RecipeID, FoodName, Instructions, Difficulty) VALUES (3, 'Beef Stew',
'Simmer beef with vegetables and broth for 2 hours.', 'Hard');
INSERT INTO Recipe(RecipeID, FoodName, Instructions, Difficulty) VALUES (4, 'Caesar Salad',
'Combine lettuce, croutons, and dressing, top with cheese.', 'Easy');
INSERT INTO Recipe(RecipeID, FoodName, Instructions, Difficulty) VALUES (5, 'Vegetable Soup',
'Boil chopped vegetables with broth for 30 minutes.', 'Easy');
```

```
-- DonateTo
INSERT INTO DonateTo(UserEmail, IngredientName, ServiceID, Date, Quantity, Unit) VALUES
('alice.johnson@gmail.com', 'Spinach', 1, '2025-10-01', 2.5, 'kg');
INSERT INTO DonateTo(UserEmail, IngredientName, ServiceID, Date, Quantity, Unit) VALUES
('brian.lee@hotmail.com', 'Chicken Breast', 2, '2025-10-03', 3, 'kg');
INSERT INTO DonateTo(UserEmail, IngredientName, ServiceID, Date, Quantity, Unit) VALUES
('catherine.smith@yahoo.com', 'Tomato', 3, '2025-10-05', 4, 'kg');
INSERT INTO DonateTo(UserEmail, IngredientName, ServiceID, Date, Quantity, Unit) VALUES
('david.kim@gmail.com', 'Beef', 4, '2025-10-07', 2, 'kg');
INSERT INTO DonateTo(UserEmail, IngredientName, ServiceID, Date, Quantity, Unit) VALUES
('emma.brown@outlook.com', 'Carrot', 5, '2025-10-09', 5, 'kg');

-- DistributeTo
INSERT INTO DistributeTo(ServiceID, FacilityID) VALUES (1, 1);
INSERT INTO DistributeTo(ServiceID, FacilityID) VALUES (2, 2);
INSERT INTO DistributeTo(ServiceID, FacilityID) VALUES (3, 3);
INSERT INTO DistributeTo(ServiceID, FacilityID) VALUES (4, 4);
INSERT INTO DistributeTo(ServiceID, FacilityID) VALUES (5, 5);

-- SoldAt
INSERT INTO SoldAt(IngredientName, StoreName, StoreAddress, Price, Availability) VALUES
('Spinach', 'GreenGrocer', '101 Market Street, Seattle', 3.49, 1);
INSERT INTO SoldAt(IngredientName, StoreName, StoreAddress, Price, Availability) VALUES
('Chicken Breast', 'FreshMart', '505 Granville St, Vancouver', 8.99, 1);
INSERT INTO SoldAt(IngredientName, StoreName, StoreAddress, Price, Availability) VALUES
('Beef', 'UrbanEats', '88 Robson St, Vancouver', 12.50, 1);
INSERT INTO SoldAt(IngredientName, StoreName, StoreAddress, Price, Availability) VALUES
('Tomato', 'DailyHarvest', '77 Queen St, Toronto', 2.20, 1);
INSERT INTO SoldAt(IngredientName, StoreName, StoreAddress, Price, Availability) VALUES
('Carrot', 'Nature’s Basket', '42 Orchard Rd, San Francisco', 2.00, 1);

-- Sells
INSERT INTO Sells(StoreName, StoreAddress, UtensilName, UtensilMaterial) VALUES
('GreenGrocer', '101 Market Street, Seattle', 'Spoon', 'Stainless Steel');
INSERT INTO Sells(StoreName, StoreAddress, UtensilName, UtensilMaterial) VALUES
('FreshMart', '505 Granville St, Vancouver', 'Fork', 'Plastic');
INSERT INTO Sells(StoreName, StoreAddress, UtensilName, UtensilMaterial) VALUES
('UrbanEats', '88 Robson St, Vancouver', 'Knife', 'Bamboo');
INSERT INTO Sells(StoreName, StoreAddress, UtensilName, UtensilMaterial) VALUES
('DailyHarvest', '77 Queen St, Toronto', 'Chopsticks', 'Wood');
INSERT INTO Sells(StoreName, StoreAddress, UtensilName, UtensilMaterial) VALUES ('Nature’s
Basket', '42 Orchard Rd, San Francisco', 'Plate', 'Ceramic');
```

```
-- Uses
INSERT INTO Uses(FoodName, UtensilName, UtensilMaterial) VALUES ('Spaghetti Bolognese', 'Fork', 'Plastic');
INSERT INTO Uses(FoodName, UtensilName, UtensilMaterial) VALUES ('Chicken Stir Fry', 'Chopsticks', 'Wood');
INSERT INTO Uses(FoodName, UtensilName, UtensilMaterial) VALUES ('Beef Stew', 'Spoon', 'Stainless Steel');
INSERT INTO Uses(FoodName, UtensilName, UtensilMaterial) VALUES ('Caesar Salad', 'Fork', 'Plastic');
INSERT INTO Uses(FoodName, UtensilName, UtensilMaterial) VALUES ('Vegetable Soup', 'Spoon', 'Stainless Steel');

-- UsedIn
INSERT INTO UsedIn(IngredientName, RecipeID, FoodName, Unit, Quantity) VALUES ('Beef', 3, 'Beef Stew', 'g', 300);
INSERT INTO UsedIn(IngredientName, RecipeID, FoodName, Unit, Quantity) VALUES ('Tomato', 1, 'Spaghetti Bolognese', 'g', 200);
INSERT INTO UsedIn(IngredientName, RecipeID, FoodName, Unit, Quantity) VALUES ('Chicken Breast', 2, 'Chicken Stir Fry', 'g', 250);
INSERT INTO UsedIn(IngredientName, RecipeID, FoodName, Unit, Quantity) VALUES ('Spinach', 4, 'Caesar Salad', 'g', 50);
INSERT INTO UsedIn(IngredientName, RecipeID, FoodName, Unit, Quantity) VALUES ('Carrot', 5, 'Vegetable Soup', 'g', 80);

-- Favorites
INSERT INTO Favorites(UserEmail, RecipeID, FoodName) VALUES ('alice.johnson@gmail.com', 1, 'Spaghetti Bolognese');
INSERT INTO Favorites(UserEmail, RecipeID, FoodName) VALUES ('brian.lee@hotmail.com', 2, 'Chicken Stir Fry');
INSERT INTO Favorites(UserEmail, RecipeID, FoodName) VALUES ('catherine.smith@yahoo.com', 3, 'Beef Stew');
INSERT INTO Favorites(UserEmail, RecipeID, FoodName) VALUES ('david.kim@gmail.com', 4, 'Caesar Salad');
INSERT INTO Favorites(UserEmail, RecipeID, FoodName) VALUES ('emma.brown@outlook.com', 5, 'Vegetable Soup');
```

9. In this project, we do not and will not use any AI tools to help us work on the assignment.